# Lab Session Week 7

## Task 1: Selection sort

```
1: SelectionSort(A)
2:     for i = 1..A.length-1
3:         s = i
4:         for j = i+1..A.length
5:             if A[j] < A[s]
7:                 s = j
8:         swap(A[i], A[s])
```

1. Try out the algorithm with cards or paper and pencil!
2. Explain in your own words how the algorithm works!
3. How does the part on lines 3–7 relate to the `minimum()` function from last weeks lab?
4. Give an invariant for the outer loop!
5. What are the best and worst cases for selection sort in terms of runtime, and how do they differ?

### Question 2

Selection sort works by swapping the smallest among the numbers that have not yet been sorted to the first position in the unsorted part of the array. Specifically, $A[i..n]$ is the part that is not yet sorted. The inner loop finds the location $s$ of the smallest element this part and line 8 then swaps the smallest element to the first position in the unsorted part.

### Question 3

Lines 3–7 are essentially the same as the `minimum()` function, except that we do not track and return the minimum value but the index of the array element containing the minimum value. This gives us slightly more information, because we can look up the value directly if we know the index, but not the other way around.

### Question 4

$A[1..i-1]$ contains the $i-1$ smallest elements in $A$ in sorted order and $A$ contains the same elements as at the start.

Note the following:

- Upon initialization, we have $i = 1$ and thus the invariant refers to $A[1..0]$, which is an empty subarray. Statements about empty arrays are always true (except if they claimed that the array is not empty), so the invariant holds on initialization.
- Upon termination, the invariant guarantees that $A[1..n-1]$ contains the $n-1$ smallest elements of $A$ in sorted order. Therefore, $A[n]$ must contain the largest element in $A$ and we can conclude that the entire array is sorted.

### Question 5

For each $i$, the inner loop will be executed $n - i$ times independent of the data in $A$. Therefore, the runtime will be $\Theta(n^2)$ in all cases. If the data are in sorted order, line 7 will never be executed and the swap operation on line 8 needs to do nothing. In that sense sorted data is a best case, but with the same scaling behavior as the worst case.

## Task 2: How fast do functions grow, and how much can we get done?

Consider the following functions, which all may describe runtimes $T(n)$ of algorithms

$$\lg n\,,\ \sqrt{n}\,,\ n\,,\ n\lg n\,,\ n^2\,,\ n^3\,,\ 2^n\,,\ n! \tag{1}$$

1. Evaluate each function for a few different values of $n$. Choose values for which you can do the math reasonably easily with paper and pencil or a simple calculator to get some "feeling" for the numbers.
2. Find out how large a problem we can solve in a given time.
   A. Assume that $T(n)$ gives the number of operations required to solve the problem for size $n$.
   B. Assume that a single operation takes $1\ \mathrm{ns}$.
   C. For each of the functions above, which problem size can we solve in $1\ \mathrm{ms}$, $1\ \mathrm{s}$, $1\ \mathrm{min}$, $1\ \mathrm{h}$, $1$ day, ..., $1$ century?
   D. Try a back-of-the envelope calculation for a small number of cases first.
   E. Write a script to tabulate all functions and durations.

In [1]:
```
import pandas as pd
import math
from scipy.optimize import bisect
```

In [2]:
```
def protect_f(func, t):
    """Return func(t) or NaN if exception is raised."""
    try:
        return func(t)
    except ArithmeticError:
        return math.nan

def format_result(n):
    """Format results nicely in LaTeX format."""
```

```
    if math.isnan(n):
        return 'NaN'
    elif n < 10000:
        return '${:.0f}$'.format(round(n))
    else:
        m, e = '{:.1e}'.format(n).split('e+')
        e = e.lstrip('0')
        return '$' + m + ' \\times 10^{' + e + '}$'
```

In [3]:
```
ops_per_sec = 1e9

# Operations we can do in given time intervals
n_ops = {'1 µs': 0.001*0.001*ops_per_sec,
         '1 ms': 0.001*ops_per_sec,
         '1 s': 1 * ops_per_sec,
         '1 min': 60*ops_per_sec,
         '1 hour': 60*60*ops_per_sec,
         '1 day': 24*60*60*ops_per_sec,
         '1 month': 30*24*60*60*ops_per_sec,
         '1 year': 365*24*60*60*ops_per_sec,
         '1 century': 100*365*24*60*60*ops_per_sec}

# Map t = T(n) to n = T^-1(t)
funcs = {r'$\lg n$': lambda t: 2**t,
         r'$\sqrt{n}$': lambda t: t**2,
         r'$n$': lambda t: t,
         r'$n\lg n$': lambda t: bisect(lambda n: n*math.log2(n)-t, 1, t),
         r'$n^2$': lambda t: t**0.5,
         r'$n^3$': lambda t: t**(1/3),
         r'$2^n$': lambda t: math.log2(t),
         r'$n!$': lambda t: bisect(lambda n: math.gamma(n+1)-t, 1, 170)
        }
```

The following cell shows a way to build the entire table using a nested comprehension and a bit of Pandas to create the entire table with just a single expression. I consider it an interesting exercise to try to express tasks such as building a table like this as a single expression.

In [9]:
```
df = pd.DataFrame.from_dict(
    {lbl: [protect_f(T_inv, t) for t in n_ops.values()] for lbl, T_inv in fu
    orient='index', columns=n_ops.keys())
```

In [10]:
```
df.style.set_properties(width='8em').format(format_result)
```

Out[10]:

| | 1 µs | 1 ms | 1 s | 1 min | 1 hour | 1 day | 1 month |
|---|---|---|---|---|---|---|---|
| $\lg n$ | $1.1 \times 10^{301}$ | NaN | NaN | NaN | NaN | NaN | NaN |
| $\sqrt{n}$ | $1.0 \times 10^{6}$ | $1.0 \times 10^{12}$ | $1.0 \times 10^{18}$ | $3.6 \times 10^{21}$ | $1.3 \times 10^{25}$ | $7.5 \times 10^{27}$ | $6.7 \times 10^{30}$ |
| $n$ | $1000$ | $1.0 \times 10^{6}$ | $1.0 \times 10^{9}$ | $6.0 \times 10^{10}$ | $3.6 \times 10^{12}$ | $8.6 \times 10^{13}$ | $2.6 \times 10^{15}$ |
| $n \lg n$ | $140$ | $6.3 \times 10^{4}$ | $4.0 \times 10^{7}$ | $1.9 \times 10^{9}$ | $9.9 \times 10^{10}$ | $2.1 \times 10^{12}$ | $5.7 \times 10^{13}$ |
| $n^2$ | $32$ | $1000$ | $3.2 \times 10^{4}$ | $2.4 \times 10^{5}$ | $1.9 \times 10^{6}$ | $9.3 \times 10^{6}$ | $5.1 \times 10^{7}$ |
| $n^3$ | $10$ | $100$ | $1000$ | $3915$ | $1.5 \times 10^{4}$ | $4.4 \times 10^{4}$ | $1.4 \times 10^{5}$ |
| $2^n$ | $10$ | $20$ | $30$ | $36$ | $42$ | $46$ | $51$ |
| $n!$ | $6$ | $9$ | $12$ | $14$ | $15$ | $17$ | $18$ |

- For $\lg n$, the results for 1 ms and longer are too large to be represented by floating point numbers in Python.
- Note the qualitative difference in scaling between polynomial ($n$, $n^2$, $n^3$) scaling and exponential ($2^n$) and factorial ($n!$) scaling!
- An example of an algorithm with $T(n) = \lg n$ is binary search on a sorted array.

In [ ]: